SAKARYA ÜNİVERSİTESİ
Bulut Bilişim Dersi ödev raporu

3d Malzeme paylaşma görüntüleme projesi

Ad Soyad: Faruk Adem
Numara: Y255012061
Ders: Bulut Bilişim
Öğretim Üyesi: Dr. Öğr. Üyesi Baran Kaynak
Tarih: 19/04/2026

1. GİRİŞ

Günümüzde platform fark etmeksizin hemen hemen her proje bir bulut çözümüne ihtiyaç duyuyor. Bu projede, bağımsız ölçeklenebilirlik sağlayan bir mikro hizmet çözümü kullanılarak modern bulut bilişim standartlarına uygun bir backend geliştirilmiştir

1.1 Bulut Bilişim ve Çoklu Bulut Altyapıları: Akademik Bir Literatür TaramasıGiriş ve Kavramsal Çerçeve

Giriş ve Kavramsal Çerçeve

Kavramsal Çerçeve: Endüstri 4.0'ın temel taşı olan bulut bilişim, dijital dönüşüm süreçlerinde teknolojiye uyumu kolaylaştıran stratejik bir altyapıdır (Çelik, 2021). Veri ve uygulamalara internet üzerinden zaman ve mekan bağımsız erişim sağlayarak işletmelere donanım maliyetlerinde düşüş, anlık güncelleme ve esnek depolama avantajları sunar (Özyurt vd., 2024).

Servis Modelleri: Hizmetler temel olarak 5 modele (IaaS, PaaS, SaaS, CaaS, NaaS) ayrılır. Özellikle SaaS, kurulum ve fiziksel altyapı yönetimi gerektirmeden tarayıcı üzerinden kullanım kolaylığı sağlar (Çelik, 2021).

Mikro Servisler ve Konteyner Mimarisi: Geleneksel (monolitik) yapılardan, otonom çalışan mikro servis ve konteyner mimarilerine geçilmiştir (Özyurt vd., 2024). Artan uygulama sayısının yarattığı yönetim karmaşıklığını çözmek için Kubernetes gibi orkestrasyon araçları sektör standardı olmuştur.

Çoklu Bulut Yönetimi: Birden fazla bulut sağlayıcısı kullanmak yönetim karmaşıklığına yol açabilir. Bu sorunu aşmak için DevOps kültürünü destekleyen ve farklı altyapıları tek arayüzden yönetmeyi sağlayan araçlar kullanılmaktadır (Özyurt vd., 2024).

Maliyet ve Güvenlik: Kurumlar, artan veri miktarı nedeniyle pahalı şirket içi sistemler yerine bulut depolamaya yönelmektedir ve bu durum ciddi bir iktisadi esneklik sağlamaktadır (Çelik, 2021). Ayrıca, servis sağlayıcıların gelişmiş yedekleme ve katmanlı güvenlik prosedürleri bulut sistemlerini oldukça güvenilir kılmaktadır.

1.2 Projenin Tanımı ve Kapsamı: 3D Malzeme Kütüphanesi

Geliştirilen sistem, kullanıcıların 3D nesne ve malzemeleri paylaşıp indirebildiği etkileşimli bir platformdur. Sisteme e-posta ve şifre ile güvenli bir şekilde kayıt olan kullanıcılar; topluluk tarafından yüklenen malzemeleri 3D ortamda görüntüleyebilir, indirebilir ve bu materyallere puan verip yorum yapabilir. Sistemde her kullanıcı yalnızca kendi yüklediği içerikleri yönetme/silme yetkisine sahiptir. Ayrıca, paylaşılan bir malzemeye etkileşim geldiğinde sahibine anlık bildirim iletilir. Özetle bu proje, tasarımcıların ihtiyaç duydukları 3D modellere kolayca ulaşıp etkileşimde bulunabildiği yaşayan bir malzeme kütüphanesidir.

Platformun Temel Fonksiyonları:

- **Kullanıcı Yönetimi:** E-posta tabanlı güvenli kayıt ve giriş sistemi.
- **Varlık Yönetimi:** 3D malzemelerin sisteme yüklenmesi (upload), güncellenmesi ve silinmesi.
- **Etkileşim Katmanı:** Malzemeler üzerine yorum yapabilme ve topluluk geri bildirimlerini takip edebilme.
- **Bildirim Sistemi:** Paylaşılan bir varlığa yorum yapıldığında veya etkileşim geldiğinde kullanıcıyı anlık olarak bilgilendiren asenkron yapı.

Canlı Uygulama ve Erişim Bilgileri

Sistem, bulut bilişim prensiplerine uygun olarak bir sanal makine (VM) üzerinde konteynerize edilmiş halde yayına alınmıştır. Uygulamanın mikro hizmet yapısı, kullanıcı arayüzü (vitrin) üzerinden canlı olarak test edilebilmektedir.

Canlı Demo Erişimi: http://94.138.211.148:8000/vitrin/

Test Bilgisi: Sistem e-posta ve şifre ile kayıt kabul etmektedir. Tüm fonksiyonlar (yükleme, silme, yorum) canlı ortamda aktiftir.

1.3. Monolitik Yapıdan Farkı ve Projedeki Karşılığı

Geleneksel **monolitik mimaride**, bir uygulamanın tüm bileşenleri (iş mantığı, veri erişimi, dosya yönetimi vb.) tek bir kod bloğu halinde geliştirilir ve aynı ortamda çalıştırılır. Bu durum başlangıçta kolaylık sağlasa da, sistem büyüdükçe kodun yönetilmesi zorlaşır, küçük bir hata tüm uygulamanın çökmesine neden olur ve donanım kaynakları verimsiz kullanılır.

Bu projede, monolitik yapının aksine "parçala ve yönet" stratejisi benimsenmiş ve sistem **üç bağımsız mikro hizmete** bölünmüştür:

1. **FastAPI Backend Servisi:** Sistemin beyni olarak görev yapan, API isteklerini karşılayan ve iş mantığını yürüten ana merkez.
2. **PostgreSQL Veritabanı Servisi:** Kullanıcı ve sistem verilerini güvenli bir şekilde saklayan bağımsız ilişkisel veri katmanı.
3. **MinIO Nesne Depolama Servisi:** Görsel veya 3D model gibi statik dosyaları S3 standartlarında tutan izole depolama birimi.

Bu ayrım sayesinde, örneğin uygulamanın sadece dosya indirme (MinIO) kısmında bir yoğunluk yaşandığında, veritabanı veya iş mantığı servislerini boş yere kopyalamak yerine sadece depolama servisi ölçeklendirilebilmektedir. Ayrıca bir servisin (örneğin MinIO'nun) yeniden başlatılması, API Gateway görevini gören FastAPI servisini tamamen durdurmaz.

1.4. Bulut Bilişim (Cloud Computing) ve Konteynerizasyon İlişkisi

Mikro hizmetlerin işletim sisteminden ve donanımdan bağımsız olarak çalışabilmesi için Docker kullanılarak konteynerizasyon işlemi gerçekleştirilmiştir. Bu yapı sayesinde uygulamalar, adından da anlaşıldığı üzere (konteyner), kendi izole ortamlarında paketlenmiş

ve gerektiğinde farklı bir sunucu altyapısına doğrudan taşınıp anında ayağa kaldırılabilir hale getirilmiştir.

Proje kapsamındaki tüm servisler, bulut bilişim standartlarına tam uyumlu olarak yapılandırılmış ve Docker Compose aracılığıyla kurulan özel bir sanal ağ üzerinden birbirleriyle haberleşmektedir. Uygulamanın bir Sanal Özel Sunucu (VPS) üzerinde bu mimariyle hayata geçirilmesi, "üretici bağımlılığını" ortadan kaldırmaktadır. Mevcut bulut sağlayıcısında herhangi bir performans sorunu veya kısıtlama yaşanması durumunda, sistem hiçbir kod değişikliği gerektirmeden farklı bir sunucuya kolaylıkla taşınabilmekte ve donanım kaynakları (CPU, RAM) izole bir şekilde, en yüksek verimle yönetilebilmektedir.

2. MİMARİ TASARIM

Bu projede her birinin ayrı görevi olan üç farklı mikro hizmet mimarisi kullanılmıştır. Sistemde veri , nesne depolama ve iş mantığı üç farklı izole servis vardır

2.1. Hizmetlerin Açıklaması

Sistemimiz aşağıdaki üç bağımsız mikro hizmetten oluşmaktadır:

- **Backend Servisi (FastAPI):** Sistemin ana motoru ve API Gateway'i olarak görev yapar. Python tabanlı modern ve yüksek performanslı bir çatı olan FastAPI kullanılarak geliştirilmiştir. İstemciden (kullanıcı tarayıcısından) gelen HTTP isteklerini karşılar, yetkilendirme ve veri doğrulama (validation) işlemlerini gerçekleştirir ve iş mantığı kurallarını işleterek diğer servislerle (veritabanı ve depolama) koordinasyonu sağlar.
- **Veritabanı Servisi (PostgreSQL):** Sistemin yapılandırılmış veri katmanıdır. Kullanıcı bilgileri, malzeme kayıtları, açıklamalar ve dosya meta verileri gibi ilişkisel verileri kalıcı olarak saklar. Kendi izole Docker konteynerinde çalışarak backend servisinden bağımsız bir veri güvenliği sunar.
- **Nesne Depolama Servisi (MinIO):** Sistemin büyük boyutlu ve yapılandırılmamış verilerini (örneğin 3D malzeme modelleri, ürün görselleri) barındıran servisidir. S3 standartlarıyla tam uyumlu çalışan MinIO, görsel dosyaların veritabanını şişirmesini engeller ve dosya okuma/yazma performansını maksimize eder.

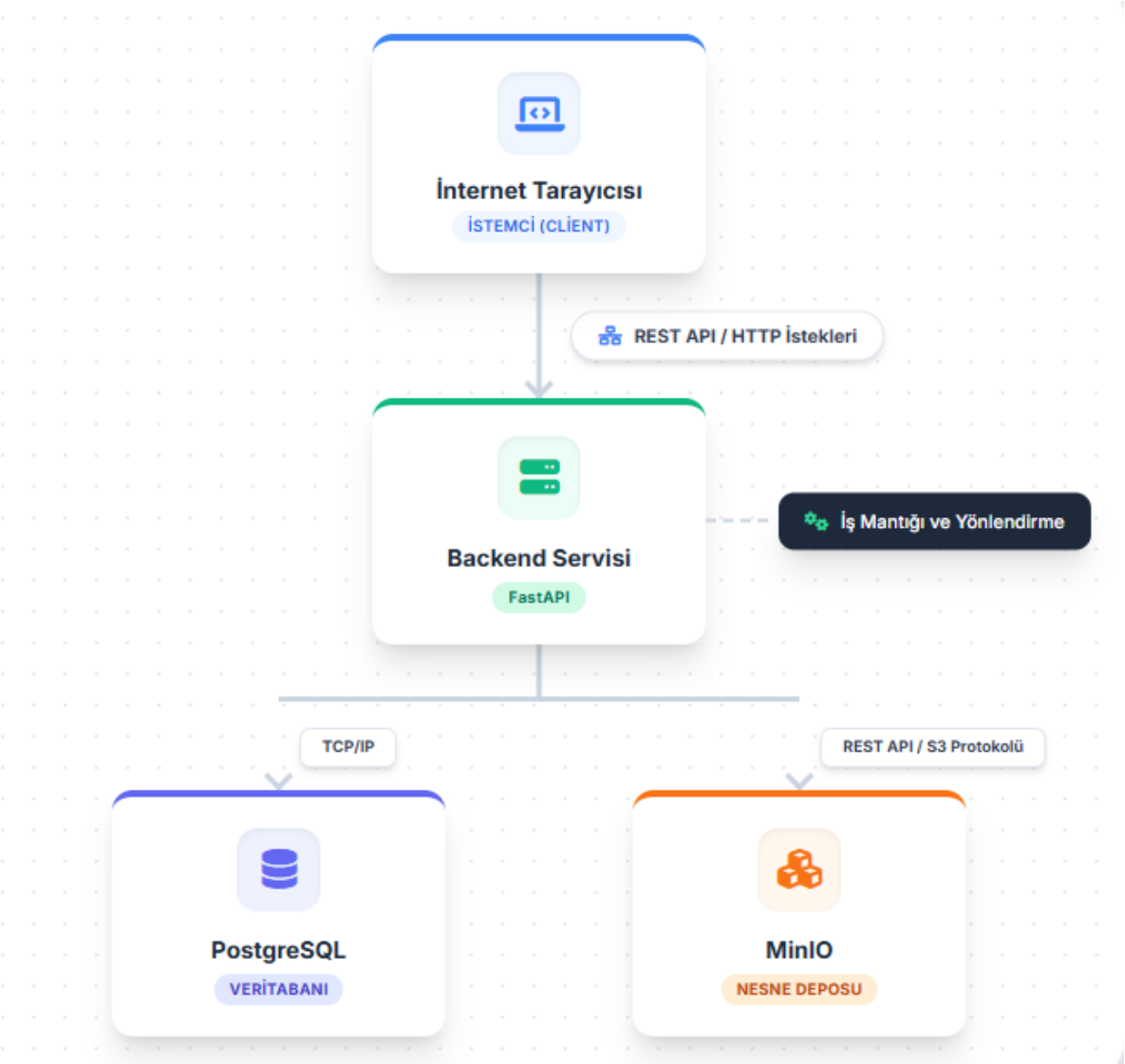
2.2. Hizmetler Arası İletişim Yapısı

- Mikro servis mimarilerinde en çok dikkat edilmesi gereken konulardan biri, servislerin birbirleriyle güvenli ve verimli bir şekilde nasıl haberleşeceğidir. Bu projede yapıyı karmaşıklaştıran mesaj kuyruklarını kullanmak yerine, daha sade ve senkron çalışan ağ protokollerini tercih ettik. Kurduğumuz sistemde tüm servisler, docker-compose.yml ile ayağa kaldırılan izole bir sanal Docker ağında (network) çalışıyor. Böylece servisler dış dünyaya tamamen kapalı bir şekilde, sadece kendi içlerinde güvenle haberleşebiliyor:

- **İstemci - Backend İletişimi:** Kullanıcı arayüzü ile sunucu arasındaki köprüyü, tarayıcı üzerinden gelen standart HTTP çağrıları oluşturuyor. Bu süreçte veri alışverişini yönetmek için GET, POST ve DELETE gibi temel REST API metotları kullanıldı.
- **Backend - Veritabanı İletişimi:** FastAPI tabanlı arka uç servisimiz, PostgreSQL veritabanıyla konuşurken SQLAlchemy kütüphanesini bir aracı katman olarak kullanıyor. Veri tutarlılığını korumak adına bu haberleşmeyi TCP/IP protokolü üzerinden senkron bir yapıyla kurguladık
- **Backend - MinIO İletişimi:** Dosya yükleme veya okuma operasyonlarında backend servisimiz doğrudan MinIO'ya bağlanıyor. Bu noktada MinIO'nun sağladığı S3 uyumlu HTTP REST API uç noktalarını kullanarak dosyaların nesne depolama katmanına aktarımı sağlanıyor.

2.3. Mimari Diyagram

Aşağıdaki şema, sistemde yer alan bileşenleri ve aralarındaki veri akış yönlerini göstermektedir:



3. UYGULAMA DETAYLARI

Bu bölümde, projenin bulut ortamına taşınmasını sağlayan konteynerizasyon süreçleri, orkestrasyon dosyalarının yapıları ve geliştirme sürecinde karşılaşılan teknik zorluklar ile üretilen çözümler detaylandırılmıştır.

1) İlk sunucu kiraladım bir Linux ubuntu sunucusu Ubuntu 24.04 LTS işletim sistemli x86_64 işlemcisine sahip 40gb depolama alanı 4g rami olan.

2)Sunucudaki güncellemeleri yaptım

3)Ssh üzerinden sunucuya bağlandım.

4)İlk Docker kurulumu yaptım

5)Sonra malzeme-projesi dosyasını oluşturdum

6)Her mikro hizmet için bir dosya açtım backend db minio adında

7)Backend in içinde :

main.py: FastAPI uygulamasını başlatır

models.py: veri tabanı tablolarını tanımlar

database.py: PostgreSQL ile backend arasındaki bağlantıdır

requirements.txt: projeye gerekli kütüphaneler.

Dockerfile: dockerın kurulum dosyası.

dosyalarını oluşturdum.

8)static/ klasörünün içine index.html i kodladım

9)db ve minio için özel docer file yazdım.

10) Bu üç hizmeti bağlamak için docker-compose.yml yazdım burda çalışmaya devam edebilmeleri için volüme ayarladım.

11) Sistemi ilk kez ayağa kaldırmaya açlışsorken mini onun sunucu işlemcisi uyumsuzluğu yüzünden hata verdi eski uyumlu bir sürümü kullanmak durumunda kaldım.

12docker-compose up --build -d ile sistemi ayağa kaldırdım

3.1. Konteynerizasyon ve Dockerfile Yapısı

Mikro hizmetlerin donanımdan ve işletim sisteminden bağımsız olarak çalışabilmesi için yukarıda detaylıca bahsettiğimiz gibi her bir servis özel olarak konteynerize edilmiştir. Proje gereksinimleri doğrultusunda her mikro hizmet için ayrı bir `Dockerfile` yapılandırılmıştır:

- **Backend Dockerfile:** FastAPI servisinin çalışacağı ortamı optimize etmek adına hafif bir Python taban imajı tercih edilmiştir. Bir diğer sebebide kiraladığım sunucunun özelliklerinin düşük olması İmaj inşası sırasında, Docker'ın katman yapısından verim almak için önce `requirements.txt` dosyası kopyalanarak bağımlılıkların yüklemesi tamamlanmış, kaynak kodlar ise bu aşamadan sonra sisteme dahil edilmiştir. Uygulamanın dış dünyaya aktarımı ve yürütülmesi için yüksek performanslı bir ASGI sunucusu olan Uvicorn kurgulanmıştır
- **Veritabanı ve Depolama Dockerfile'ları:** PostgreSQL ve MinIO servisleri için resmi imajları temel alan (sarmalayıcı) Dockerfile'lar oluşturulmuştur.

3.2. Servis Orkestrasyonu: Docker Compose

Birbirinden bağımsız olarak çalışan bu üç konteynerin, tek bir sistem gibi hareket edebilmesi için `docker-compose.yml` dosyası tasarlanmıştır. Bu dosya şu kritik görevleri üstlenmektedir:

- **Bağımlılık Yönetimi (`depends_on`):** Backend servisinin çalışmaya başlamadan önce, Veritabanı ve MinIO servislerinin tamamen ayağa kalktığından emin olunması sağlanmıştır.
- **Ağ İzolasyonu:** Tüm servisler aynı sanal köprü ağına (bridge network) dahil edilerek, birbirlerine IP adresleri yerine doğrudan servis adlarıyla (örneğin `db` veya `minio`) erişebilmeleri sağlanmıştır.
- **Çevresel Değişkenler (Environment Variables):** Veritabanı şifreleri ve MinIO erişim anahtarları gibi hassas bilgiler, Compose dosyası üzerinden dinamik olarak konteynerlere aktarılmış, böylece kod içine yazılmalarının önüne geçilmiştir.

3.3. Karşılaşılan Zorluklar ve Çözümler

Projenin geliştirilmesi ve sunucuya (VPS) aktarılması sürecinde çeşitli teknik engellerle karşılaşılmış ve bu problemlere yönelik optimum çözümler üretilmiştir:

MinIO Konteynerinin Çökmesi (Exit Code 127 Hatası):

- *Sorun:* Sistemin ilk testlerinde, nesne depolama servisi olan MinIO başlatıldığında konteynerin anında kapanarak `127` hata kodunu döndürdüğü gözlemlenmiştir. Bu durum uygulamanın dosya yükleme işlemlerini durdurmuştur.
- *Çözüm:* Hataya, kullanılan Docker imajının sürüm uyumsuzluğunun ve `command` satırında belirtilen sunucu başlatma argümanlarının eksikliğinin neden olduğu tespit edilmiştir. İmaj, daha stabil olan `quay.io/minio/minio` versiyonu ile değiştirilmiş ve konteynerin ayakta kalmasını sağlayan `server /data --console-address ":9001"` komut dizisi Compose dosyasına entegre edilerek sorun kalıcı olarak çözülmüştür.

2. **Gereksiz Dosya Yükü ve Repo Kirliliği (Git Tracking Sorunu):**
   - *Sorun:* Geliştirme ortamındaki Python sanal ortam kütüphaneleri (`venv`) ve PostgreSQL'in ham veri dosyaları (`data`), sürüm kontrol sistemine (Git) yanlışlıkla dahil edilmiş ve projenin boyutunun gereksiz yere megabaytlarca şişmesine yol açmıştır.

- *Çözüm:* İmaj optimizasyonu ve temiz bir teslimat mimarisi sağlamak amacıyla, Git'in takip belleği (index) `git rm -r --cached` komutuyla tamamen temizlenmiştir. Ardından kapsamlı bir `.gitignore` dosyası oluşturularak sadece kaynak kodların, Dockerfile'ların ve orkestrasyon dosyalarının repoda yer alması sağlanmış; projenin boyut ve yapı optimizasyonu başarıyla tamamlanmıştır.

4. ÖLÇEKLENDİRME TARTIŞMASI VE GELECEK VİZYONU

Mikro hizmet mimarilerinin en büyük avantajlarından biri, uygulamanın tamamını büyütmek yerine, sadece darbogaz yaşayan servisin kopyalarını artırarak yatayda ölçeklendirebilme yeteneğidir.

Proje sistem gereksinimleride oldukça düşük olduğu için birçok sistemde direkt çalışır.

Bu projede tasarlanan sistemin yoğun trafik altında nasıl ölçeklendirilebileceğine dair stratejiler üç ana başlık altında toplanmıştır:

4.1. İş Mantığı Katmanı (FastAPI Backend) İçin Yatay Ölçeklendirme

FastAPI servisimiz durumsuz bir yapıya sahip olduğu için yatay ölçeklendirmeye en uygun bileşendir. Geliştirme ortamında , bu servisin yükü kaldıramadığı durumlarda tek bir komutla servis sayısı artırılabilir: `docker-compose up -d --scale backend=3` Ancak bu mimarinin tam verimli çalışabilmesi için, dışarıdan gelen istekleri bu 3 farklı backend konteyneri arasında uymlu bir şekilde dağıtacak bir Yük Dengeleyici ngnix gibi bir bileşeninin sisteme API Gateway olarak eklenmesi gerekmektedir.

4.2. Veri ve Depolama Katmanlarının Ölçeklendirilmesi

Durumlu servislerin ölçeklendirilmesi, iş mantığı servislerine göre daha karışıktır:

- **Veritabanı (PostgreSQL):** Okuma ve yazma işlemlerinin yoğunluğuna göre ölçek sunucuya yetersiz kalırsa . Yazma işlemleri ana sunucuya yapılırken, sadece okuma gerektiren vitrin listeleme işlemleri kopya sunuculardan çekilerek ana veritabanının yükü hafifletilebilir.
- **Nesne Depolama (MinIO):** MinIO, doğası gereği bulut tabanlı bir depolama çözümü olduğu için birden fazla sunucu diski tek bir havuz gibi çalıştırılabilir. Bu sayede depolama kapasitesi yatayda sınırsızca büyütülebilir.

4.3. Kubernetes (K8s) ile Üretime (Production) Geçiş Planı

Mevcut mimarimiz geliştirme ve test süreçleri için Docker Compose ile optimize edilmiş olsa da, sistemin canlı ortama taşınması durumunda **Kubernetes (Minikube veya bulut sağlayıcıların K8s hizmetleri)** altyapısına geçilmesi vizyonlanmıştır.

Özellikle backend servisimizin yoğun trafik anlarında otomatik olarak çoğaltılabilmesi için gereklidir. Bu yapı sayesinde, CPU kullanımı belirli bir eşiği aştığında sistem kendi kendine yeni konteynerler üretecektir

5. SONUÇ VE DEĞERLENDİRME

Bu proje, modern bulut bilişim standartlarına uygun şekilde tasarlanmış; Docker ile konteynerize edilerek bir VPS sunucusu üzerinde yayına alınmıştır. Projenin çalışma süreci tanıtım videosunda detaylandırılmış, durdurma ve başlatma aşamaları gösterilmiştir. Sistemin web arayüzü çıktısı HTML olarak üretilmiş olup, erişim detaylarına rapor içerisinde yer verilmiştir. Mimari yapıda üç ayrı mikro hizmet kullanılmış; bu hizmetlerin teknik detayları raporda kapsamlı bir şekilde açıklanmıştır.

5.2. Konteyner Teknolojilerinin Bulut Bilişimdeki Önemi

Konteyner teknolojileri (Docker), uygulamanın altyapıdan bağımsız bir şekilde çalışmasını sağlayarak bulut bilişimin "her yerde çalışabilirlik" vaadini gerçeğe dönüştürmüştür. Geleneksel sanal makinelere (VM) kıyasla çok daha az kaynak tüketen ve saniyeler içinde başlatılabilen konteynerler; "Benim bilgisayarımda çalışıyordu, sunucuda çalışmıyor" problemini tamamen ortadan kaldırmıştır. Projede geliştirilen uygulamanın hiçbir işletim sistemi bağımlılığı olmadan doğrudan bulut sunucusuna taşınabilmesi, konteyner teknolojilerinin modern bulut bilişim mimarilerindeki vazgeçilmez rolünün en net göstergesidir.

6.KAYNAKÇA

https://docs.docker.com/compose/

https://fastapi.tiangolo.com/deployment/docker/

https://docs.min.io/enterprise/aistor-object-store/installation/container/

Çelik, K. (2021). Bulut Bilişim Teknolojileri. Bartın Üniversitesi İktisadi ve İdari Bilimler Fakültesi Dergisi, 12(24), 436-450. https://doi.org/10.47129/bartiniibf.1019898

Özyurt, H., Akıer, Y. C., & Özyurt, Ö. (2024). Çoklu bulut teknolojisi yönetimi için web tabanlı bir araç tasarımı ve geliştirilmesi. *Dicle University Journal of Engineering*, *15*(1), 77-86. https://doi.org/10.24012/dumf.1385760